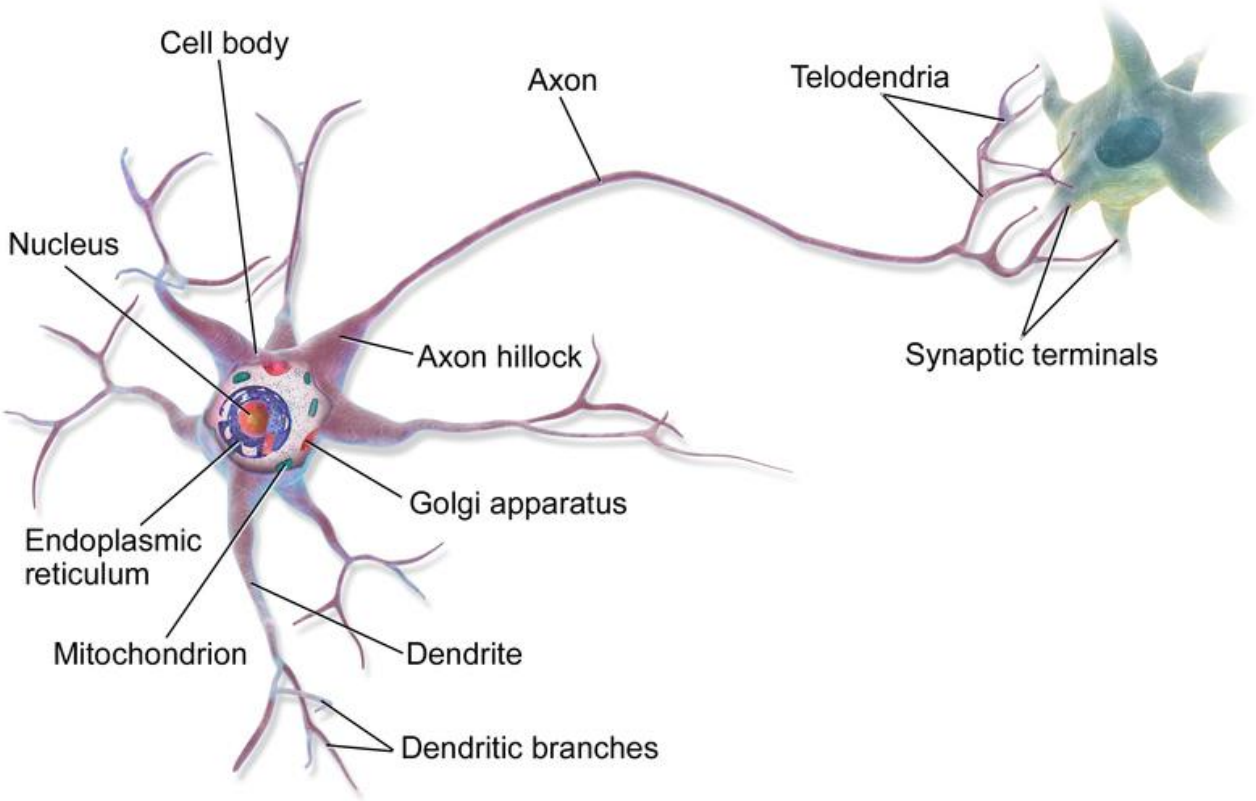
Neural Networks

Euhyun Moon, Ph.D.
Machine Learning Systems (MLSys) Lab
Computer Science and Engineering
Sogang University



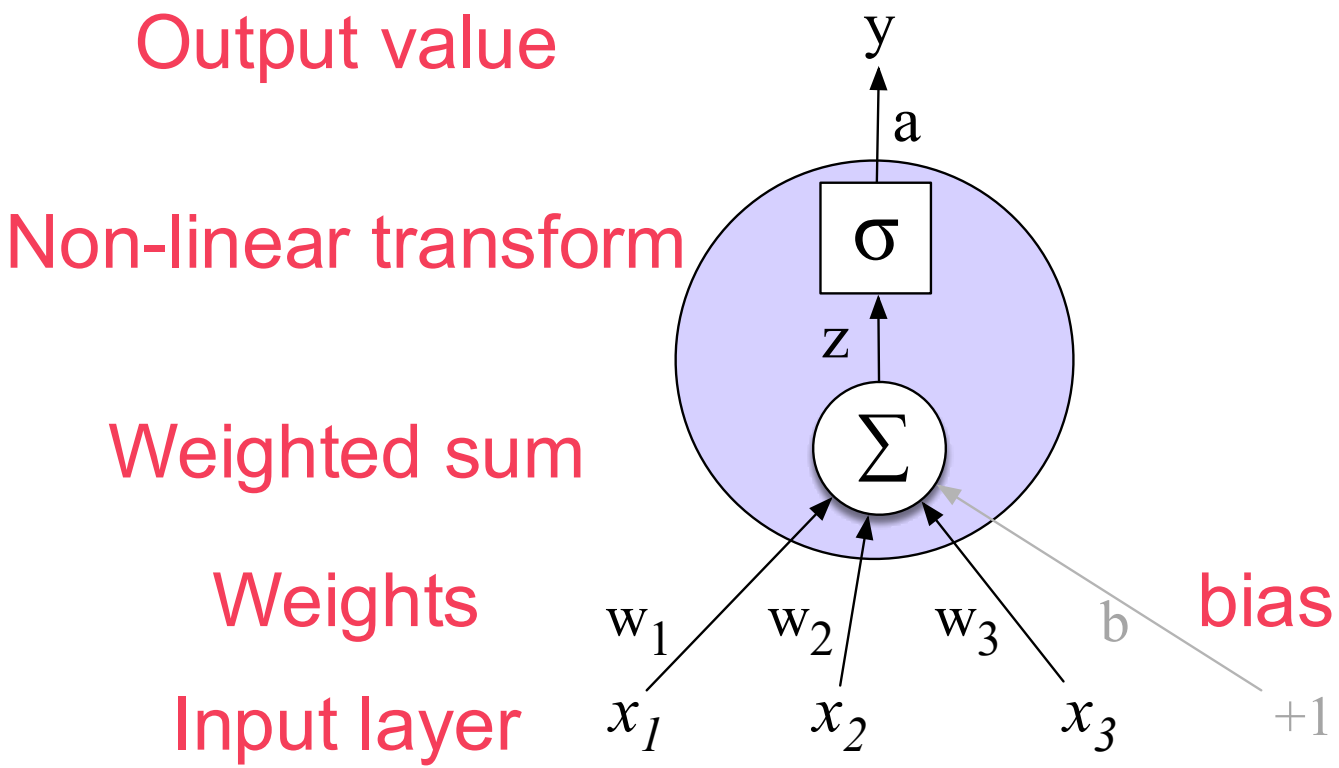
Slides adapted from D. Jurafsky: Neural Networks, https://web.stanford.edu/class/cs124, Stanford University

This is in your brain



Source: https://commons.wikimedia.org/w/index.php?curid=28761830

Neural Network Unit



Neural Unit

- Take weighted sum of inputs, plus a bias

$$z = b + \sum_i w_i x_i$$

$$z = w \cdot x + b$$

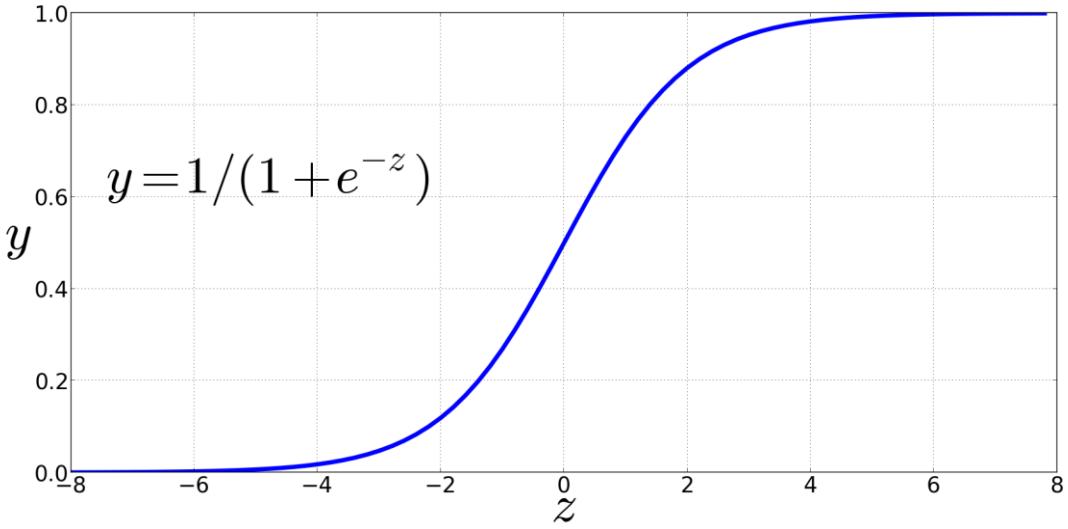
- Instead of just using z, we'll apply a nonlinear activation function f:

$$y = a = f(z)$$

Non-Linear Activation Functions

Sigmoid

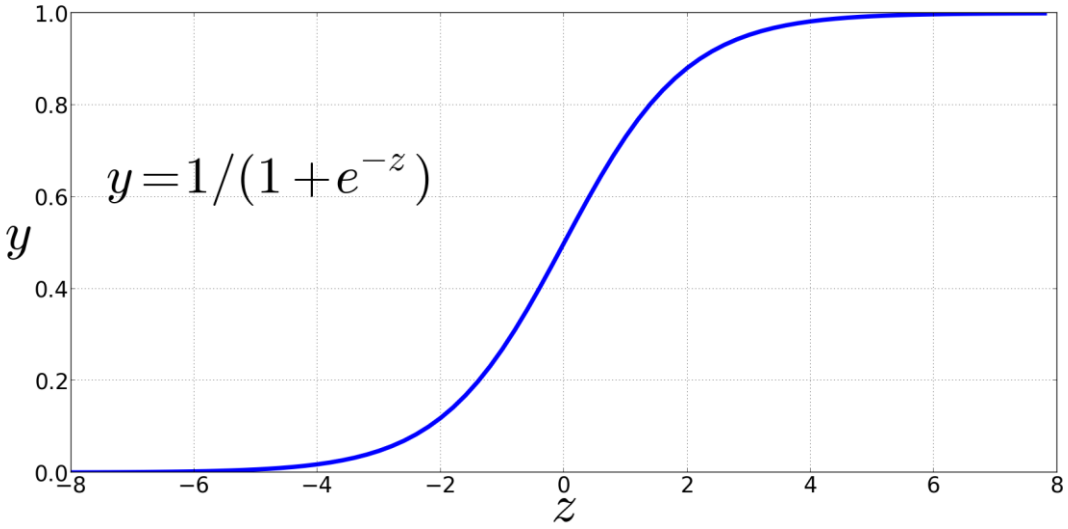
$$y = s(z) = \frac{1}{1 + e^{-z}}$$



Final Function the Unit is Computing

Sigmoid

$$y = s(z) = \frac{1}{1 + e^{-z}}$$



$$y = s(w \cdot x + b) = \frac{1}{1 + \exp(-(w \cdot x + b))}$$

Final Unit Again

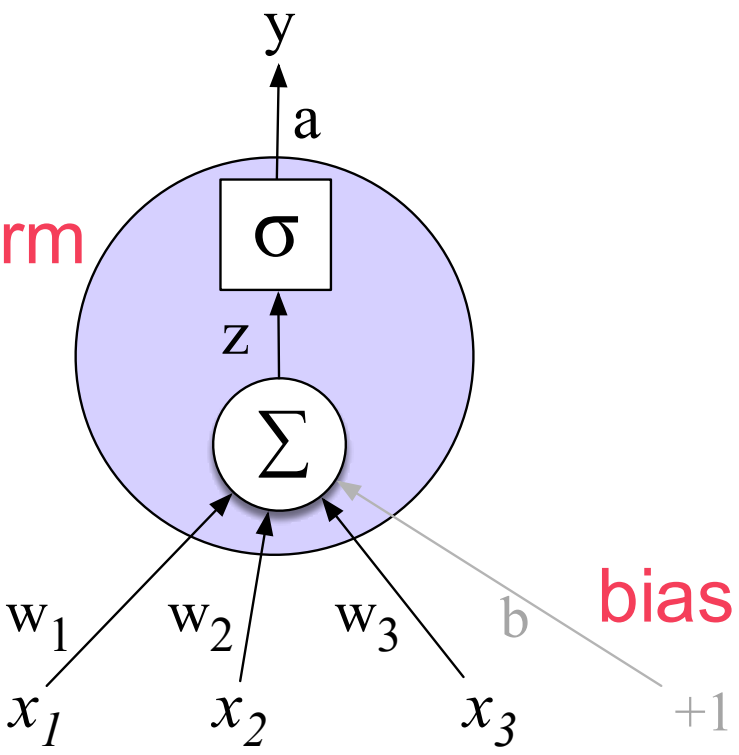
Output value

Non-linear transform

Weighted sum

Weights

Input layer



An Example

- Suppose a unit has:

$$w = [0.2, 0.3, 0.9]$$
$$b = 0.5$$

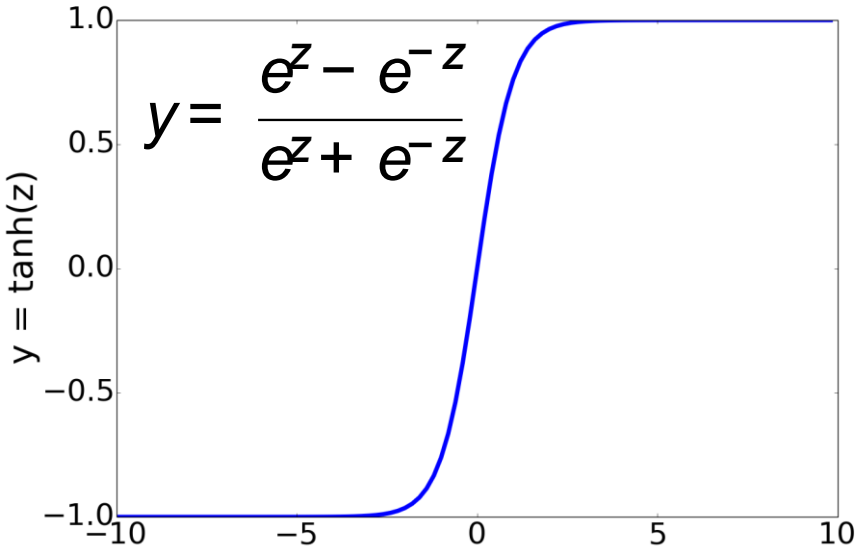
- What happens with input x:

$$x = [0.5, 0.6, 0.1]$$

$$y = s(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} =$$

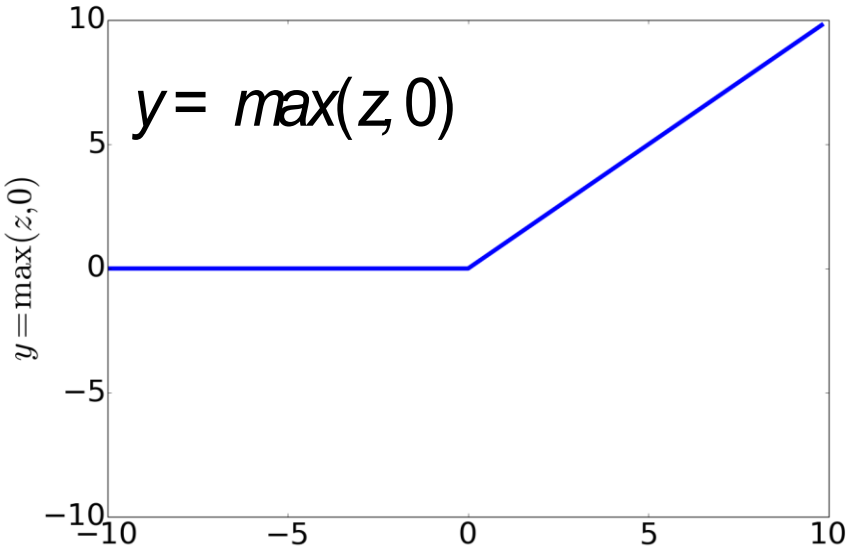
$$\frac{1}{1 + e^{-(.5 \times .2 + .6 \times .3 + .1 \times .9 + .5)}} = \frac{1}{1 + e^{-0.87}} = .70$$

Non-Linear Activation Functions Besides Sigmoid



tanh

Most Common:



ReLU
(Rectified Linear Unit)

The XOR Problem

- Can neural units compute simple functions of input?

AND			OR			XOR		
x1	x2	y	x1	x2	y	x1	x2	y
0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0

Source: Minsky and Papert 1969

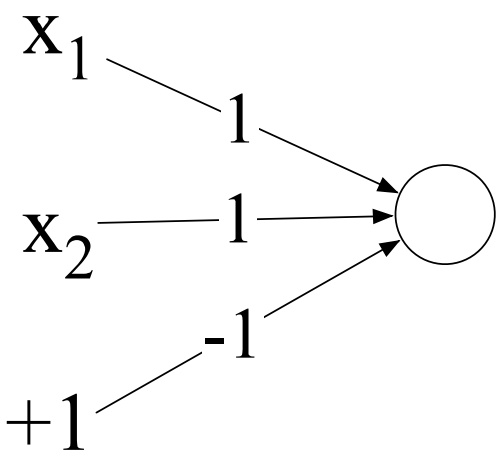
Perceptrons

- A very simple neural unit
  - Binary output (0 or 1)
  - No non-linear activation function

$$y = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases}$$

Easy to Build AND or OR with Perceptrons

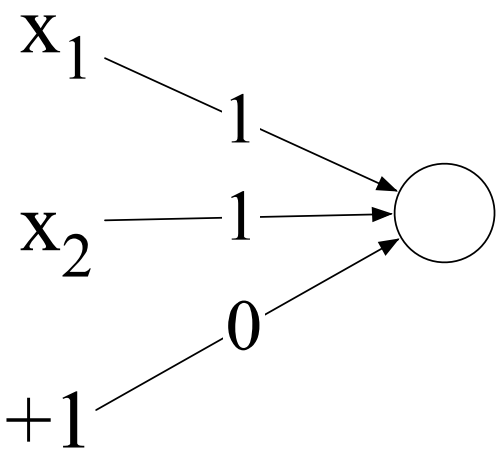
$$y = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases}$$



AND

AND

x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1



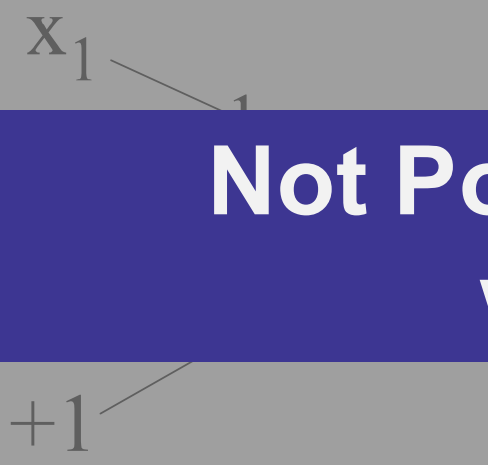
OR

OR

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	1

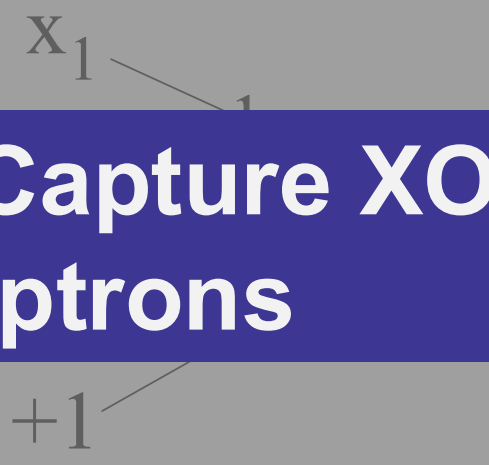
Easy to Build AND or OR with Perceptrons

$$y = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases}$$



AND

x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1



OR

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	1

Not Possible to Capture XOR with Perceptrons

Why? Perceptrons are Linear Classifiers

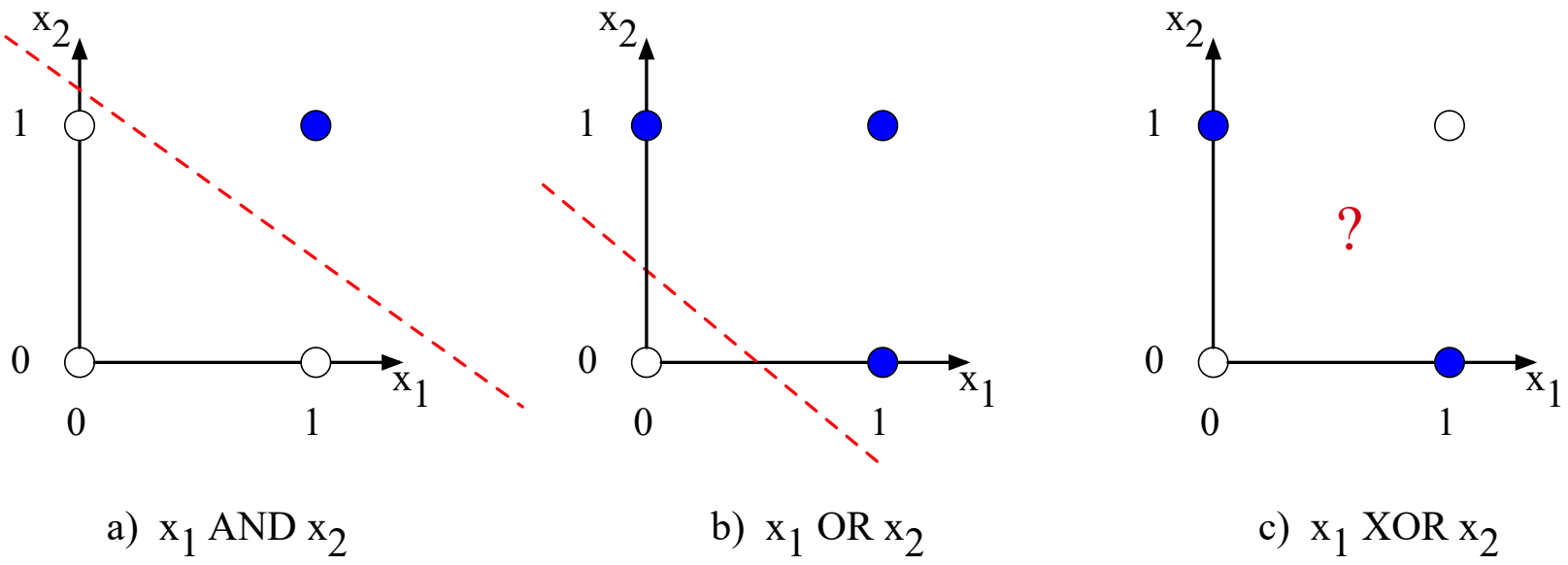
- Perceptron equation given x_1 and x_2, is the equation of a line

$$w_1 x_1 + w_2 x_2 + b = 0$$

(in standard linear format: $x_2 = (-w_1/w_2)x_1 + (-b/w_2)$)

- This line acts as a **decision boundary**
  - 0 if input is on one side of the line
  - 1 if on the other side of the line

Decision Boundaries



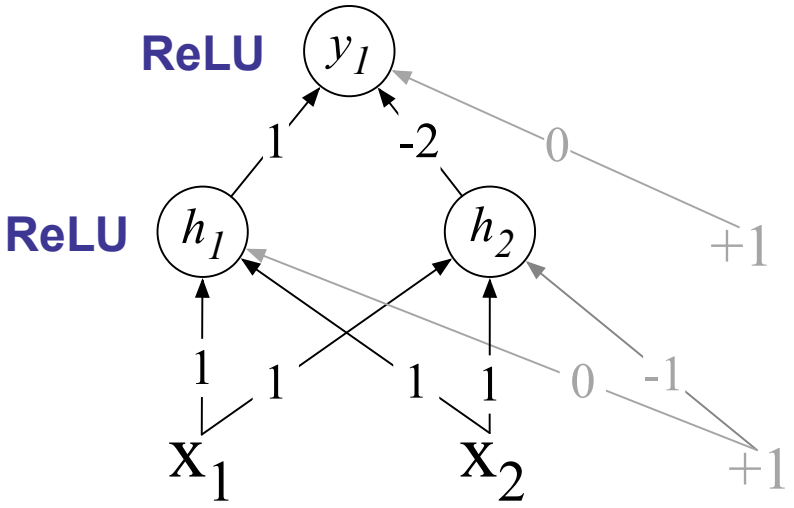
a) x_1 AND x_2 b) x_1 OR x_2 c) x_1 XOR x_2

XOR is not a **linearly separable** function!

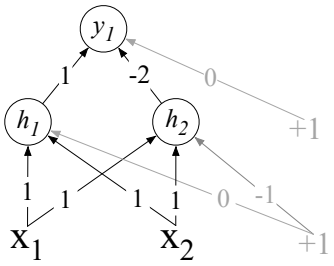
Solution to the XOR Problem

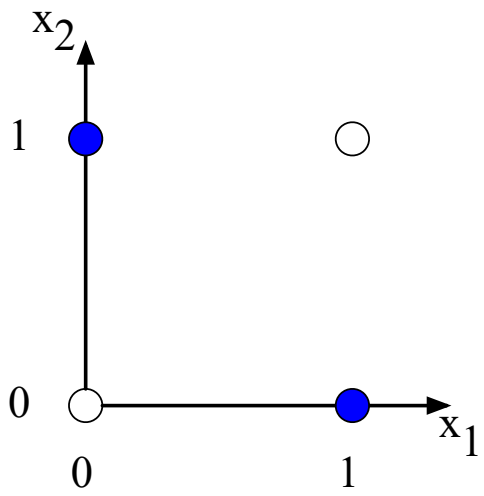
- XOR **can't** be calculated by a single perceptron
- XOR **can** be calculated by a layered network of units

XOR		
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

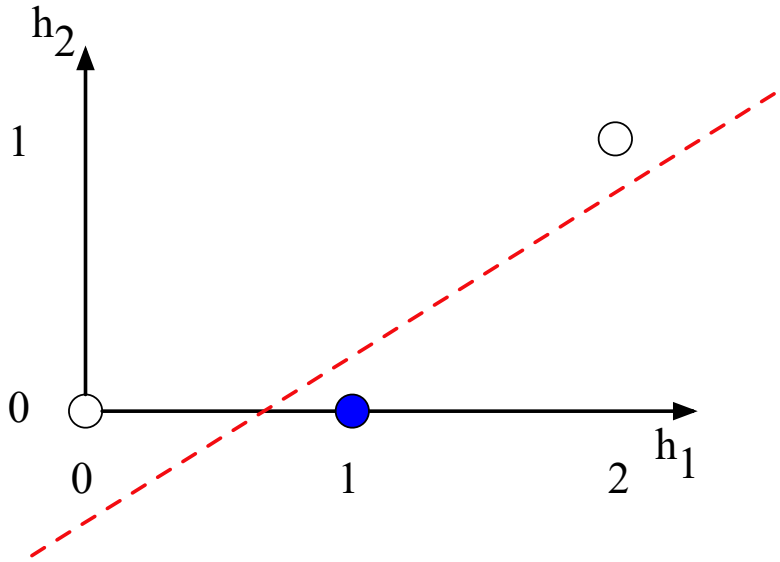


The Hidden Representation *h*





a) The original *x* space

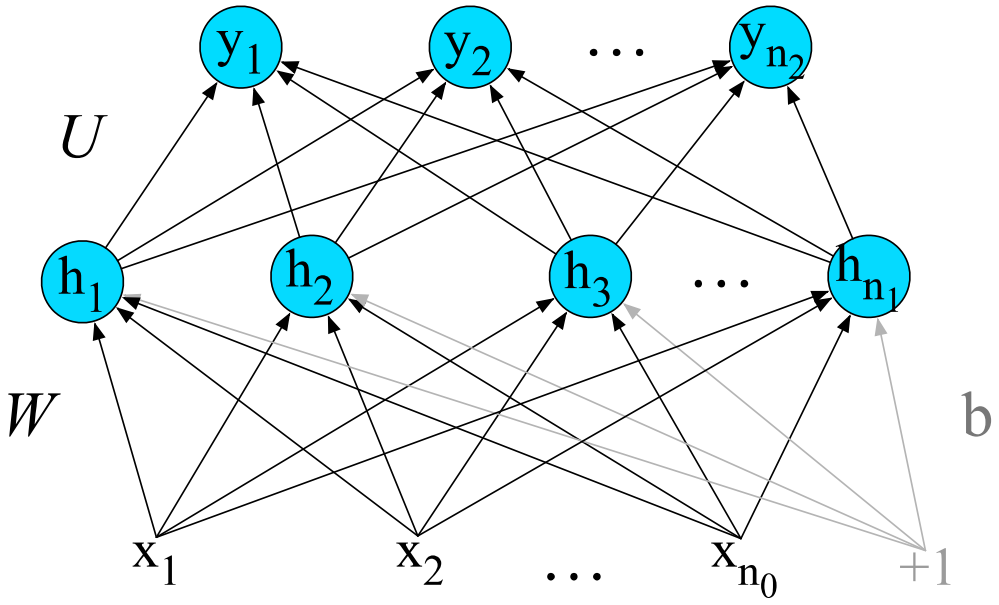


b) The new (linearly separable) *h* space

(With learning: hidden layers will learn to form useful representations)

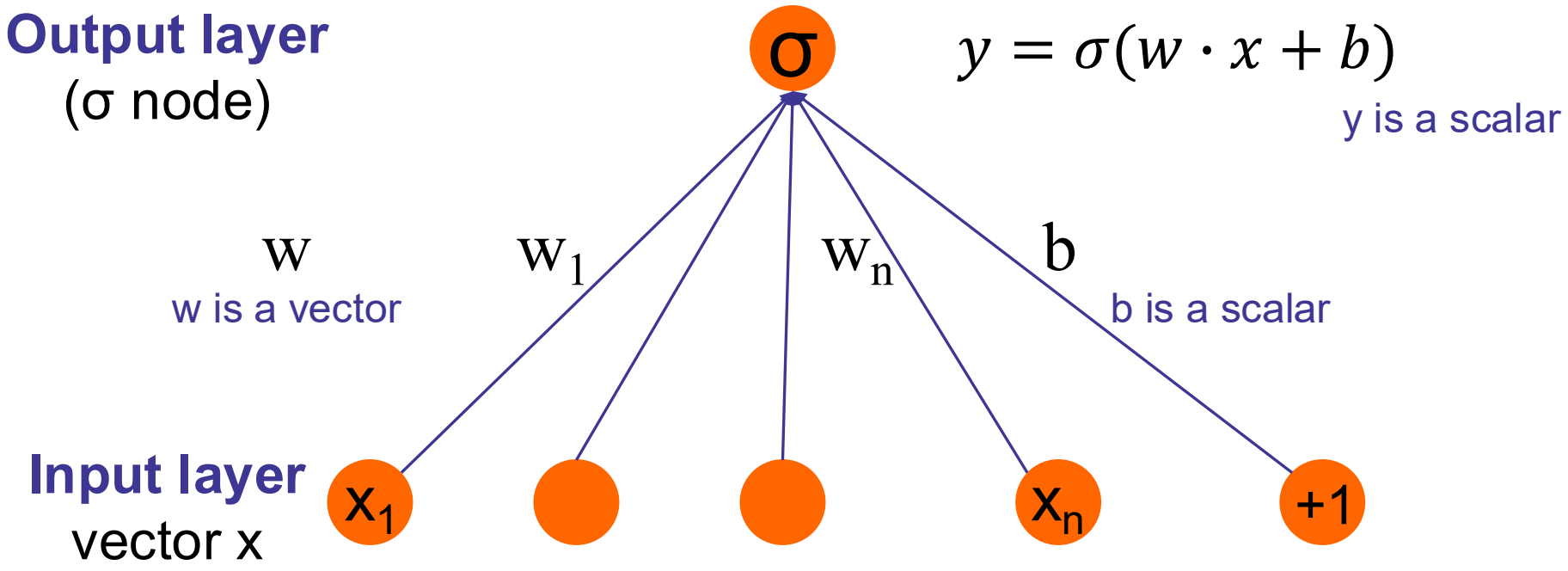
Feedforward Neural Networks

- Can also be called **multi-layer perceptrons** (or **MLPs**) for historical reasons



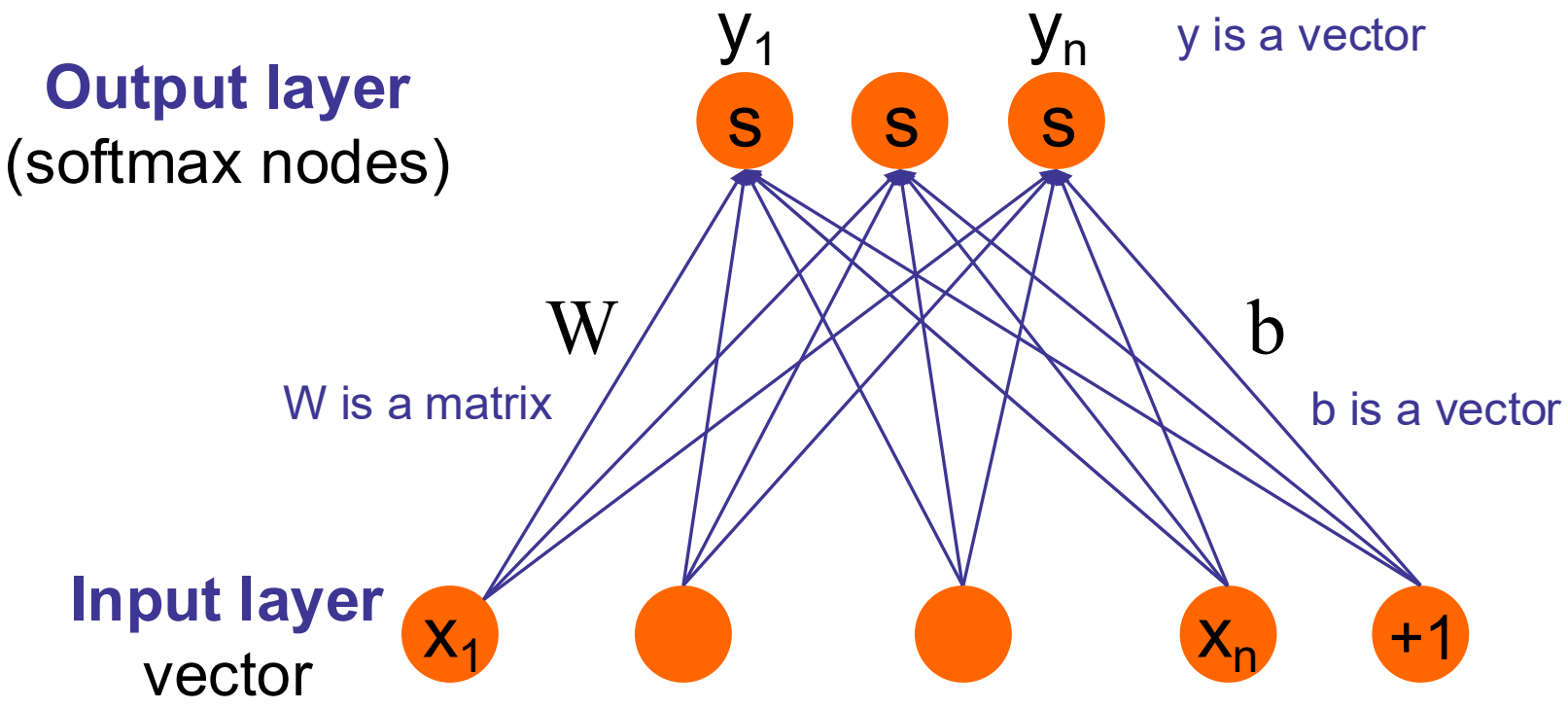
Binary Logistic Regression as a 1-Layer Network

(we don't count the input layer in counting layers!)



Multinomial Logistic Regression as a 1-Layer Network

Fully connected single layer network



Softmax: a generalization of sigmoid

- For a vector z of dimensionality k, the softmax is:

$$\text{softmax}(z) = \left[\frac{\exp(z_1)}{\sum_{i=1}^{k} \exp(z_i)}, \frac{\exp(z_2)}{\sum_{i=1}^{k} \exp(z_i)}, \ldots, \frac{\exp(z_k)}{\sum_{i=1}^{k} \exp(z_i)} \right]$$

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^{k} \exp(z_j)} \quad 1 \leq i \leq k$$

- Example:

$$z = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$$

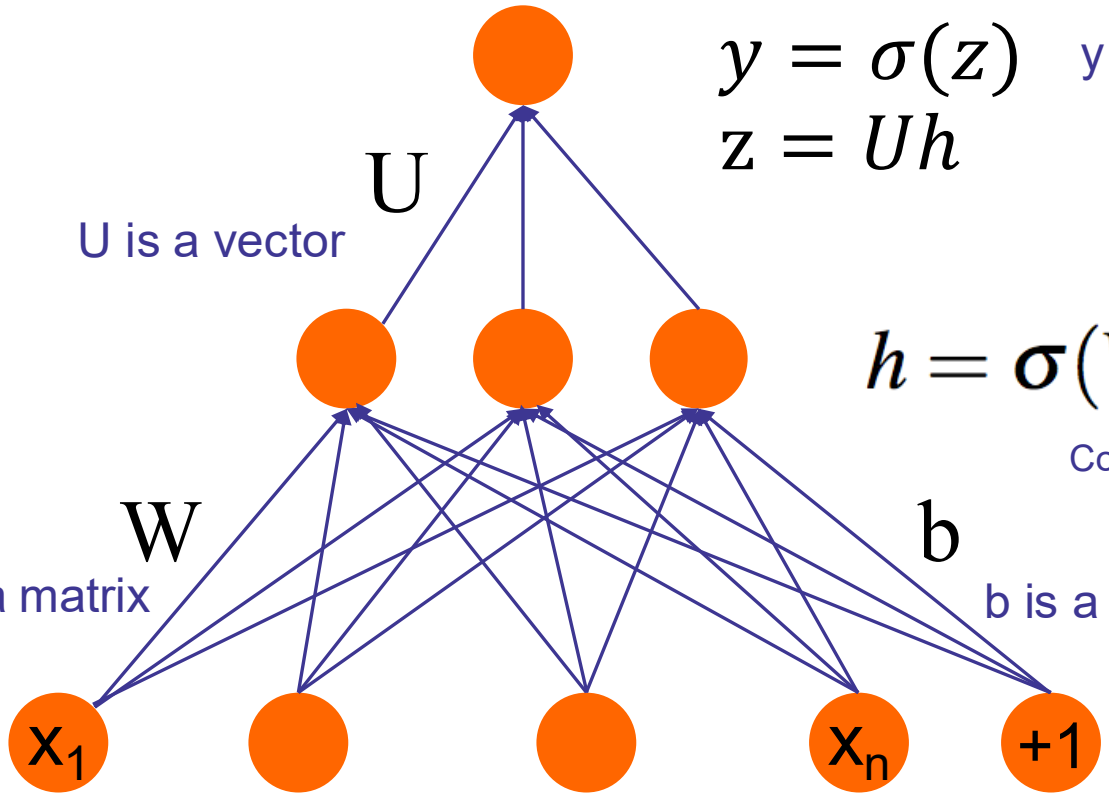
$$\text{softmax}(z) = [0.055, 0.090, 0.006, 0.099, 0.74, 0.010]$$

Two-Layer Network with Scalar Output

Output layer
(σ node)

hidden units
(σ node)

Input layer
vector



$$y = \sigma(z)$$ y is a scalar

$$z = Uh$$

$$h = \sigma(Wx + b)$$

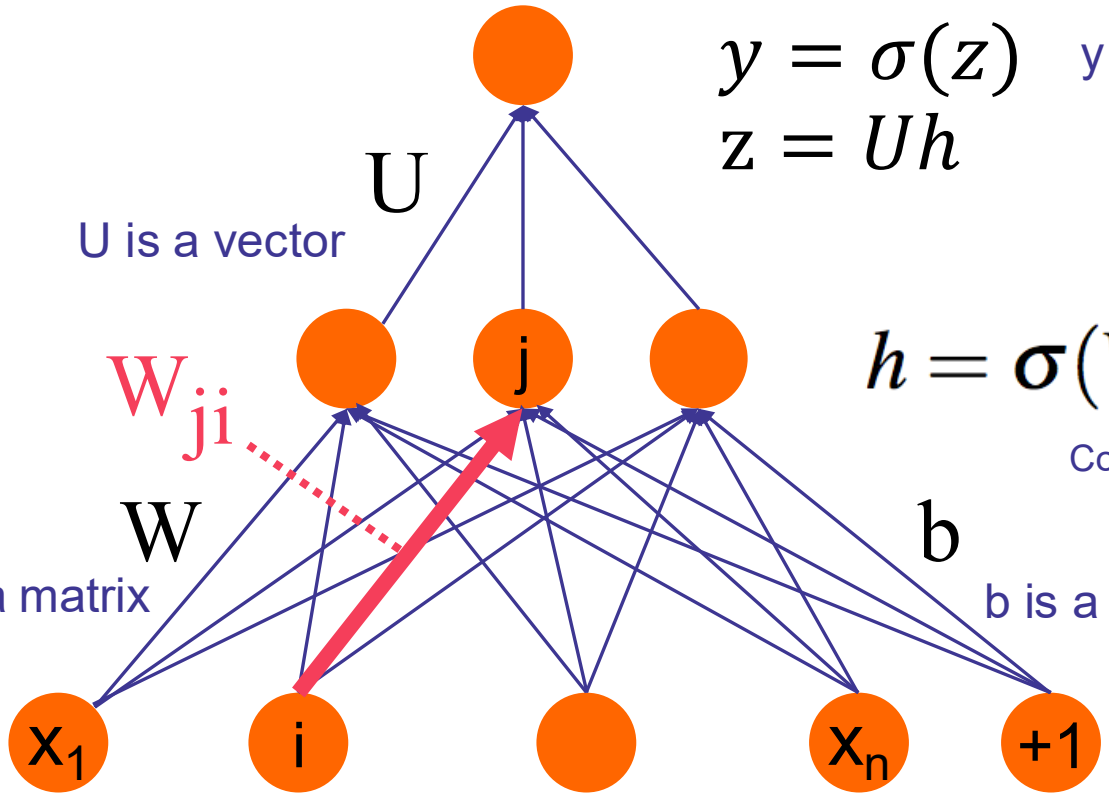
Could be ReLU or tanh

Two-Layer Network with Scalar Output

Output layer
(σ node)

hidden units
(σ node)

Input layer
vector



$$y = \sigma(z)$$ y is a scalar

$$z = Uh$$

$$h = \sigma(Wx + b)$$

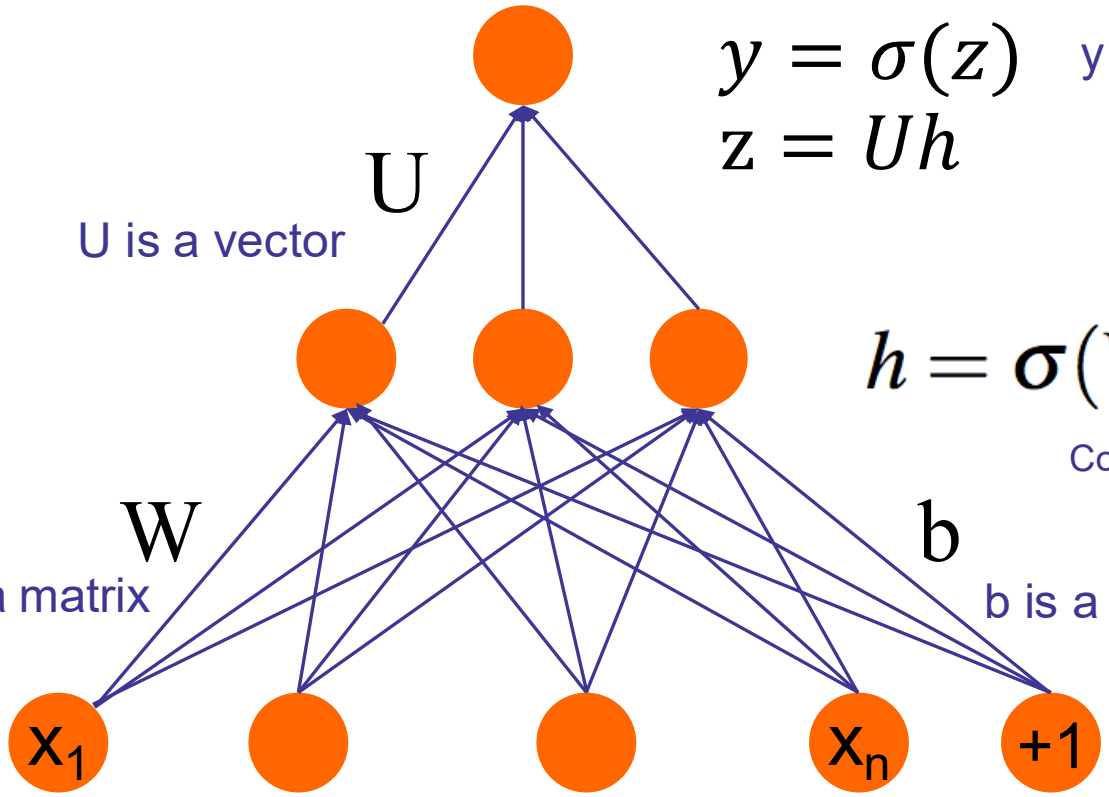
Could be ReLU or tanh

Two-Layer Network with Scalar Output

Output layer
(σ node)

hidden units
(σ node)

Input layer
vector



$$y = \sigma(z)$$ y is a scalar

$$z = Uh$$

$$h = \sigma(Wx + b)$$

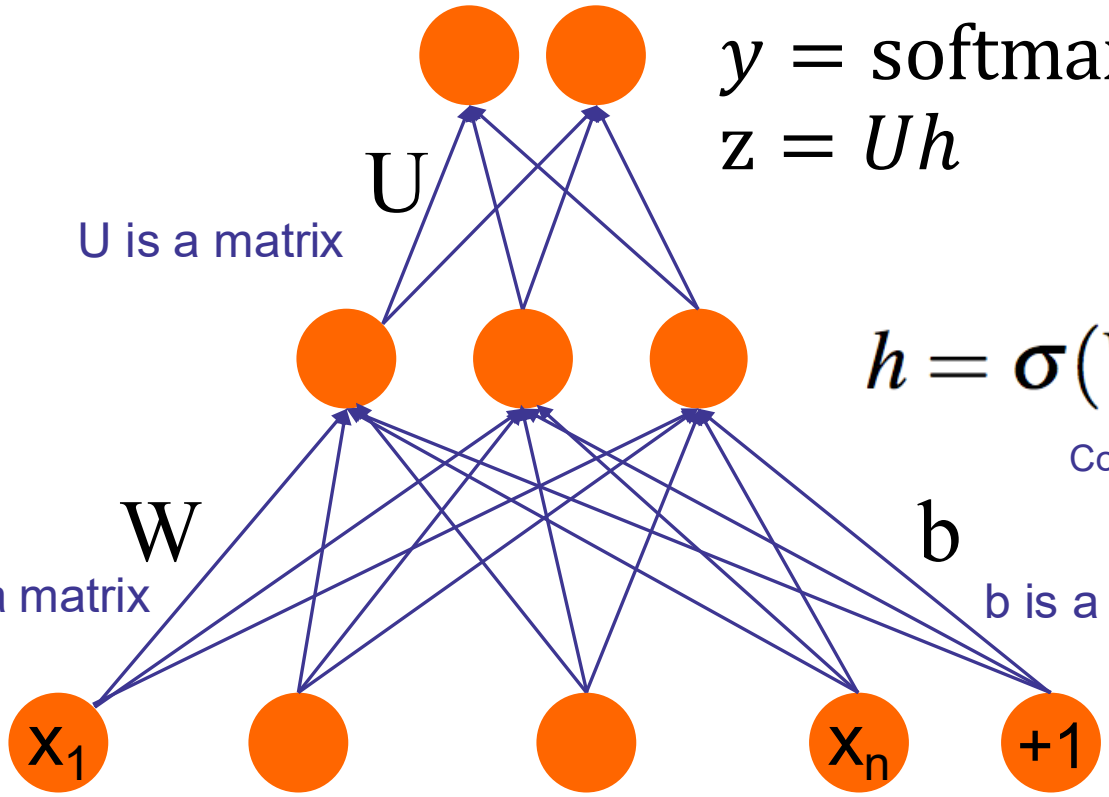
Could be ReLU or tanh

Two-Layer Network with Softmax Output

Output layer
(σ node)

hidden units
(σ node)

Input layer
vector



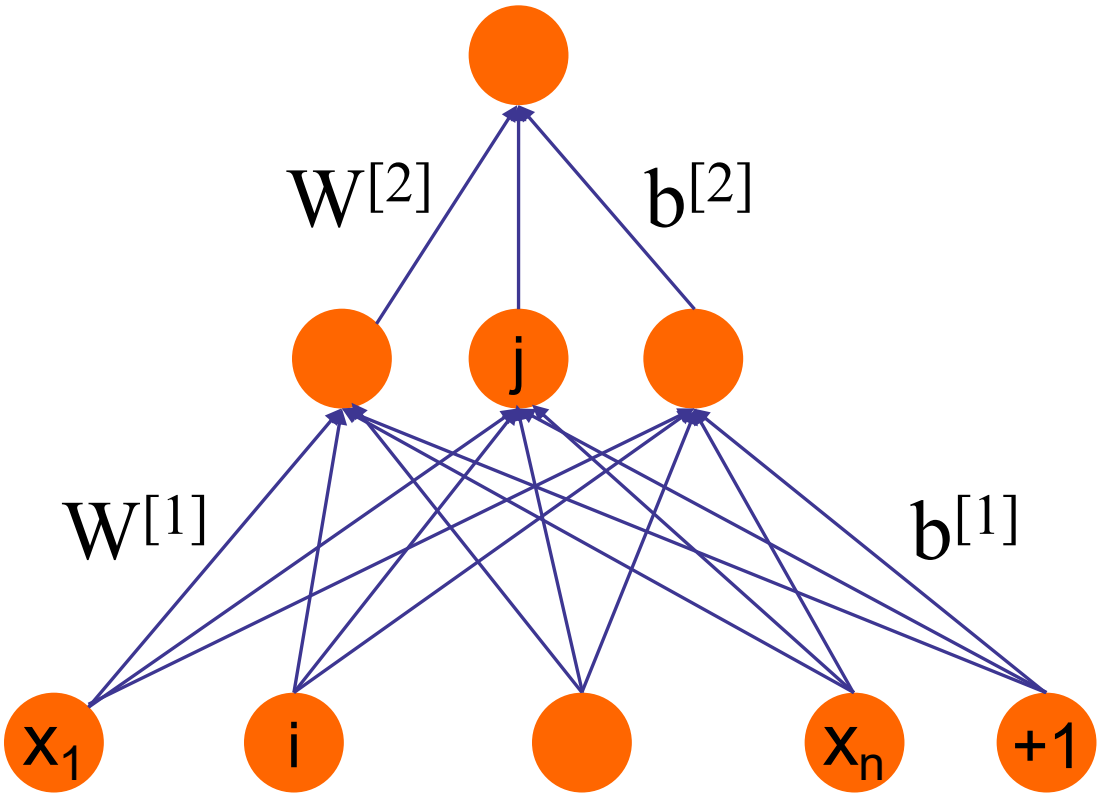
$$y = \text{softmax}(z)$$
$$z = Uh$$

y is a vector

$$h = \sigma(Wx + b)$$

Could be ReLU or tanh

Multi-Layer Notion



$$y = a^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]})$$ sigmoid or softmax

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

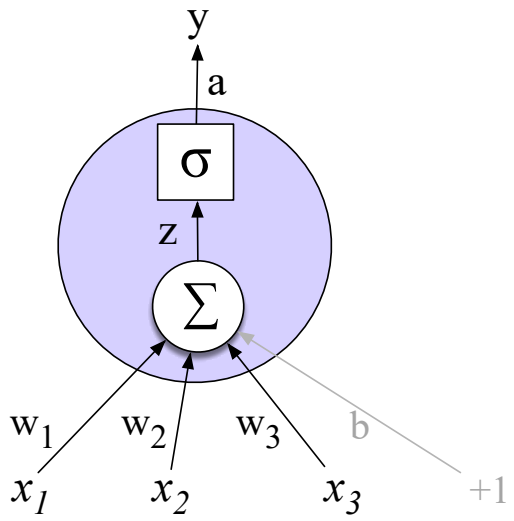
$$a^{[1]} = g^{[1]}(z^{[1]})$$ ReLU

$$z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$$

$a^{[0]}$

Multi-Layer Notion

$$
\begin{aligned}
z^{[1]} &= W^{[1]} a^{[0]} + b^{[1]} \\
a^{[1]} &= g^{[1]}(z^{[1]}) \\
z^{[2]} &= W^{[2]} a^{[1]} + b^{[2]} \\
a^{[2]} &= g^{[2]}(z^{[2]}) \\
\hat{y} &= a^{[2]}
\end{aligned}
$$



for i **in** 1..n

$$
\begin{aligned}
z^{[i]} &= W^{[i]} a^{[i-1]} + b^{[i]} \\
a^{[i]} &= g^{[i]}(z^{[i]}) \\
\hat{y} &= a^{[n]}
\end{aligned}
$$

Replacing the Bias Unit

- Let's switch to a notation without the bias unit
- Just a notational change
  1. Add a dummy node $a_0=1$ to each layer
  2. Its weight w_0 will be the bias
  3. So input layer $a^{[0]}_0=1$,
     - And $a^{[1]}_0=1$, $a^{[2]}_0=1$,…

Replacing the Bias Unit

Instead of:

$$x = x_1, x_2, \ldots, x_{n0}$$

$$h = \sigma(Wx + b)$$

$$h_j = \sigma\left(\sum_{i=1}^{n_0} W_{ji}x_i + b_j\right)$$

We'll do this:

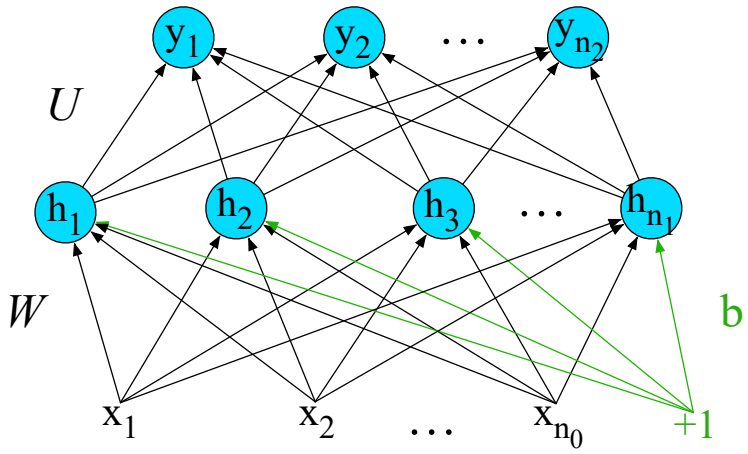
$$x = x_0, x_1, x_2, \ldots, x_{n0}$$

$$h = \sigma(Wx)$$

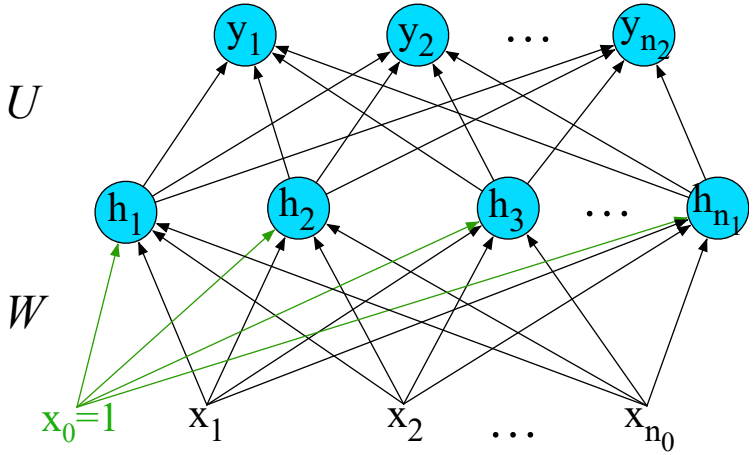
$$h_j = \sigma\left(\sum_{i=0}^{n_0} W_{ji}x_i\right)$$

Replacing the Bias Unit

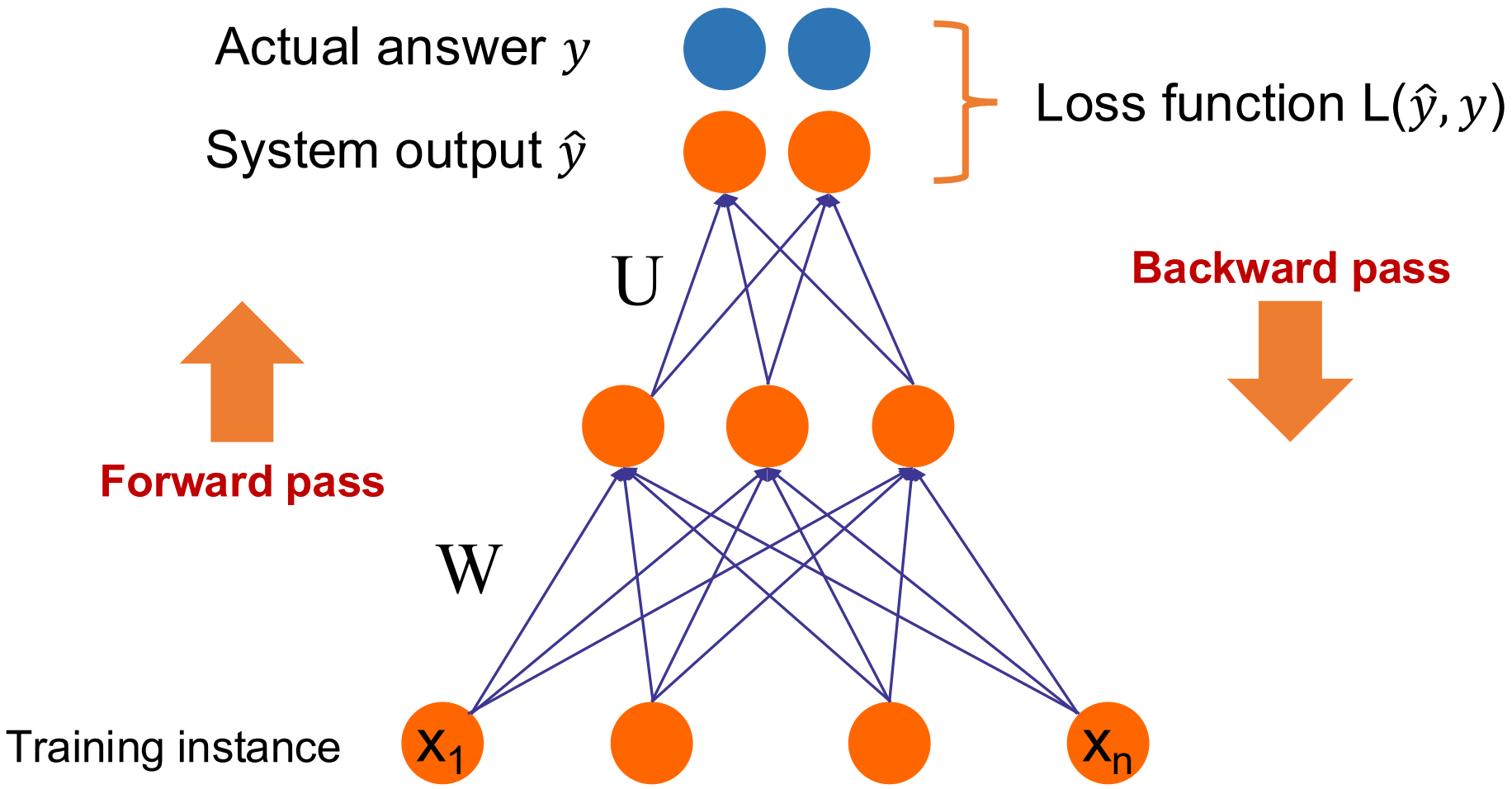
Instead of:



We'll do this:



Intuition: Training a 2-Layer Network



Intuition: Training a 2-Layer Network

- For every training tuple (x, y)
  - Run *forward* computation to find our estimate $\hat{y}$
  - Run *backward* computation to update weights:
    - For every output node
      - Compute loss L between true y and the estimated $\hat{y}$
      - For every weight w from hidden layer to the output layer
        - Update the weight
    - For every hidden node
      - Assess how much blame it deserves for the current answer
      - For every weight w from input layer to the hidden layer
        - Update the weight

Loss Function for Binary Logistic Regression

- A measure for how far off the current answer is to the right answer
- Cross entropy loss for logistic regression:

$$L_{\text{CE}}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1-y)\log(1-\hat{y})]$$

$$= -[y \log \sigma(w \cdot x + b) + (1-y)\log(1-\sigma(w \cdot x + b))]$$

Gradient Descent for Weight Updates

- Use the derivative of the loss function with respect to weights $\frac{d}{dw} L(f(x; w), y)$
- To tell us how to adjust weights for each training item
  - Move them in the opposite direction of the gradient

$$w^{t+1} = w^t - h \frac{d}{dw} L(f(x; w), y)$$

  - For logistic regression

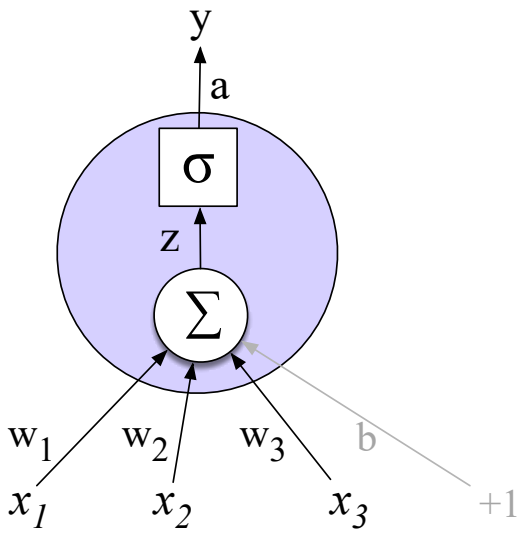
$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y] x_j$$

Where did that derivative come from?

- Using the **chain rule**! $f(x) = u(v(x))$

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}$$

- Intuition



Derivative of the weighted sum

Derivative of the Activation

Derivative of the Loss

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_i}$$

How can I find that gradient for every weight in the network?

- These derivatives on the prior slide only give the updates for one weight layer: the last one!
- What about deeper networks?
  - Lots of layers, different activation functions?
- Solution:
  - Even more use of the chain rule!
  - Computation graphs and backward differentiation!

Why Computation Graphs

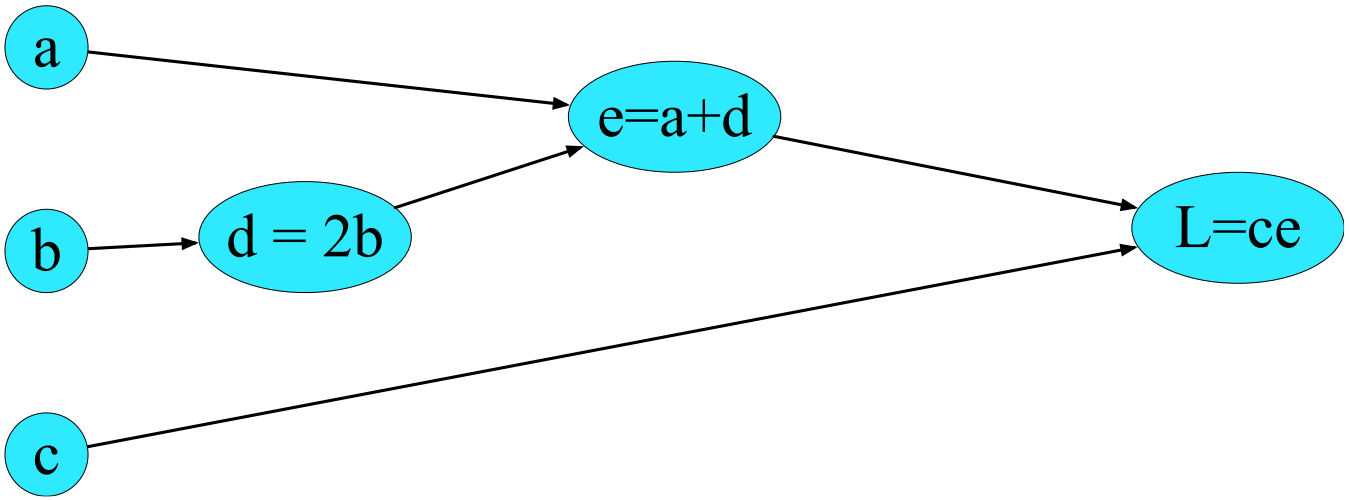
- For training, we need the derivative of the loss with respect to each weight in every layer of the network
  - But the loss is computed only at the very end of the network!
- Solution: **error backpropagation** (Rumelhart, Hinton, Williams, 1986)
  - **Backprop** is a special case of **backward differentiation**
  - Which relies on **computation graphs**

- Computation Graphs:
  - A computation graph represents the process of computing a mathematical expression

Example: L(a, b, c) = c(a + 2b)

Computations:

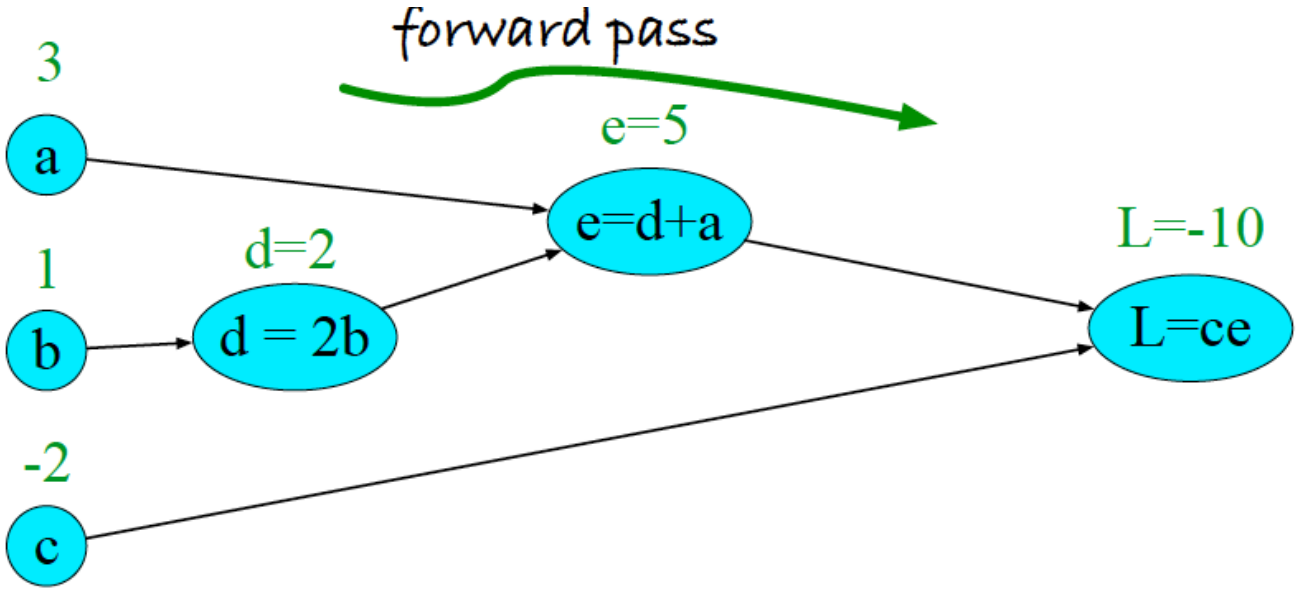
$$d = 2 * b$$
$$e = a + d$$
$$L = c * e$$



Example: L(a, b, c) = c(a + 2b)

Computations:

$$d = 2 * b$$
$$e = a + d$$
$$L = c * e$$



Backwards Differentiation in Computation Graphs

- The importance of the computation graph comes from the backward pass
- This is used to compute the derivatives that we'll need for the weight update

Example: L(a, b, c) = c(a + 2b)

$$d = 2 * b$$
$$e = a + d$$
$$L = c * e$$

- We want: $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, and $\frac{\partial L}{\partial c}$

The derivative $\frac{\partial L}{\partial a}$, tells us how much a small change in *a* affects *L*

The Chain Rule

- Computing the derivative of a composite function:

$$f(x) = u(v(x)) \qquad \frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}$$

$$f(x) = u(v(w(x))) \qquad \frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dw} \cdot \frac{dw}{dx}$$

Example: L(a, b, c) = c(a + 2b)

$$
\begin{aligned}
d &= 2 * b \\
e &= a + d \\
L &= c * e
\end{aligned}
$$

$$
\begin{aligned}
\frac{\partial L}{\partial c} &= e \\
\frac{\partial L}{\partial a} &= \frac{\partial L}{\partial e}\frac{\partial e}{\partial a} \\
\frac{\partial L}{\partial b} &= \frac{\partial L}{\partial e}\frac{\partial e}{\partial d}\frac{\partial d}{\partial b}
\end{aligned}
$$

Example: L(a, b, c) = c(a + 2b)

$$
\begin{aligned}
d &= 2 * b \\
e &= a + d \\
L &= c * e
\end{aligned}
$$

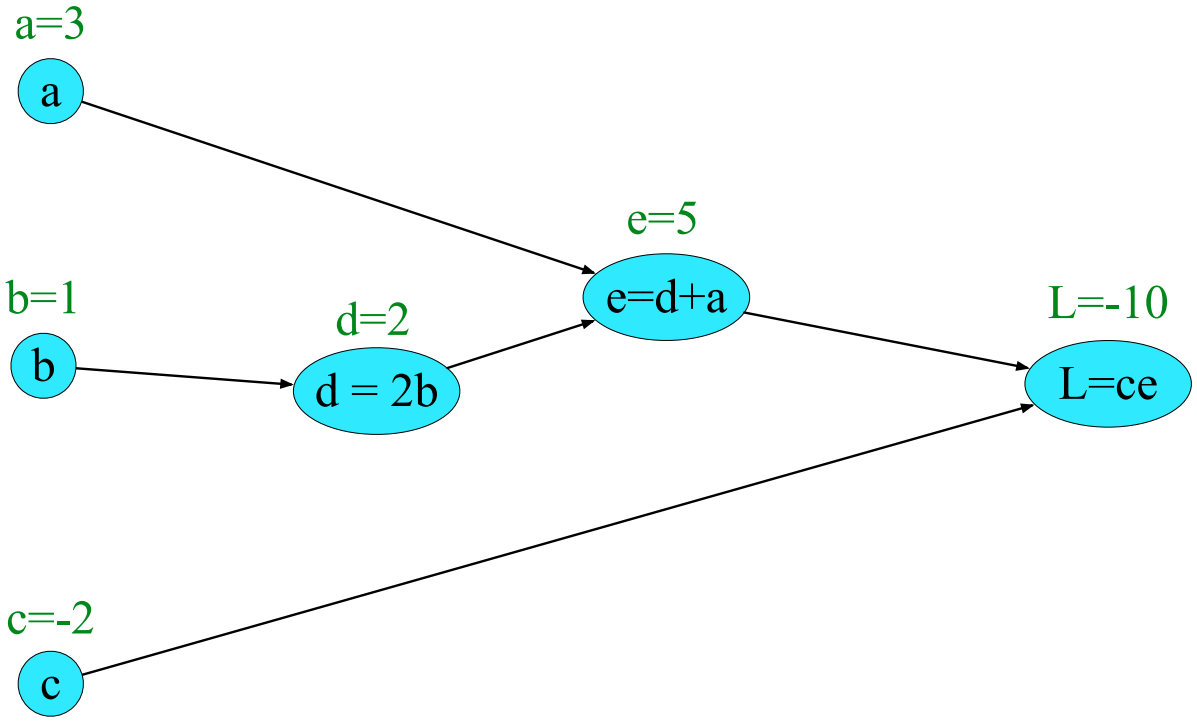
$$
\begin{aligned}
\frac{\partial L}{\partial a} &= \frac{\partial L}{\partial e}\frac{\partial e}{\partial a} \\
\frac{\partial L}{\partial b} &= \frac{\partial L}{\partial e}\frac{\partial e}{\partial d}\frac{\partial d}{\partial b}
\end{aligned}
$$

$$
\begin{aligned}
L = ce &: \quad \frac{\partial L}{\partial e} = c, \frac{\partial L}{\partial c} = e \\
e = a + d &: \quad \frac{\partial e}{\partial a} = 1, \frac{\partial e}{\partial d} = 1 \\
d = 2b &: \quad \frac{\partial d}{\partial b} = 2
\end{aligned}
$$

Example: L(a, b, c) = c(a + 2b)

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e}\frac{\partial e}{\partial a}$$
$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e}\frac{\partial e}{\partial d}\frac{\partial d}{\partial b}$$

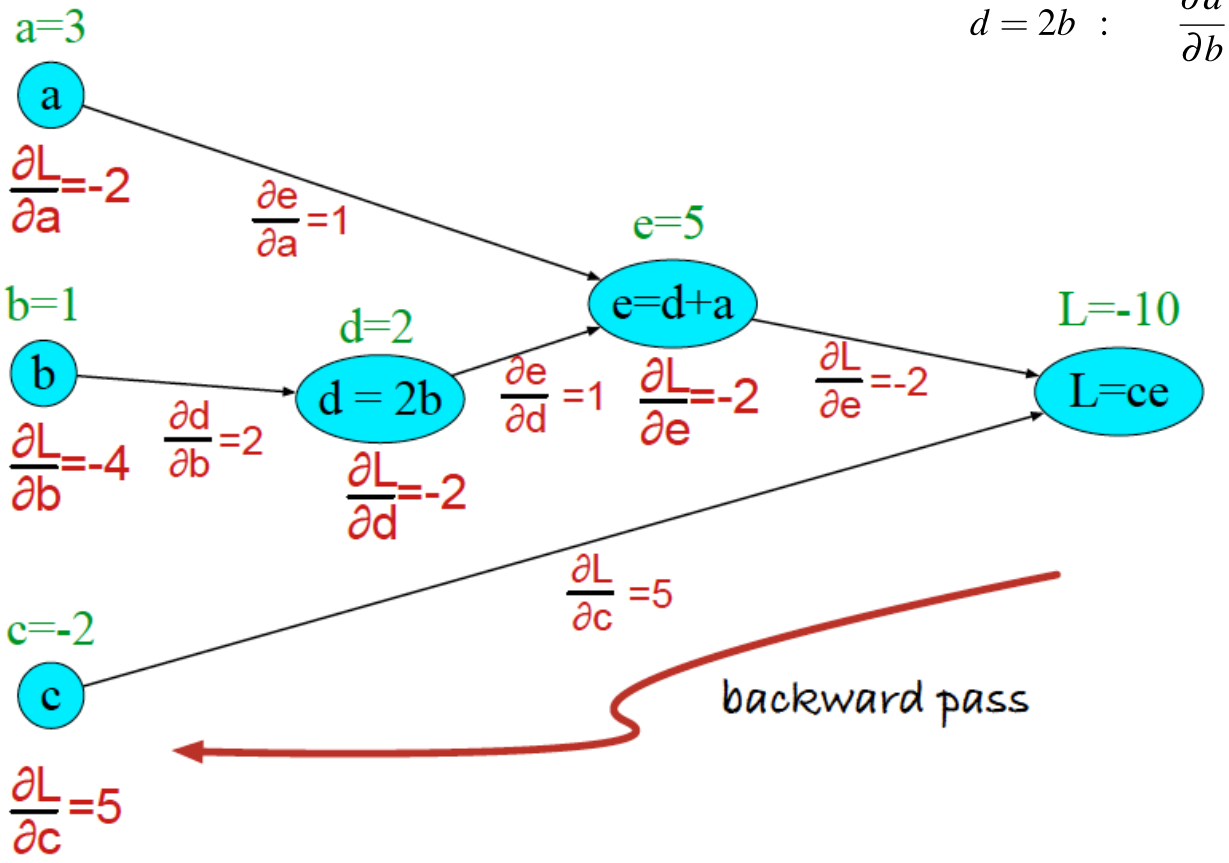
$$L = ce : \quad \frac{\partial L}{\partial e} = c, \frac{\partial L}{\partial c} = e$$
$$e = a + d : \quad \frac{\partial e}{\partial a} = 1, \frac{\partial e}{\partial d} = 1$$
$$d = 2b : \quad \frac{\partial d}{\partial b} = 2$$



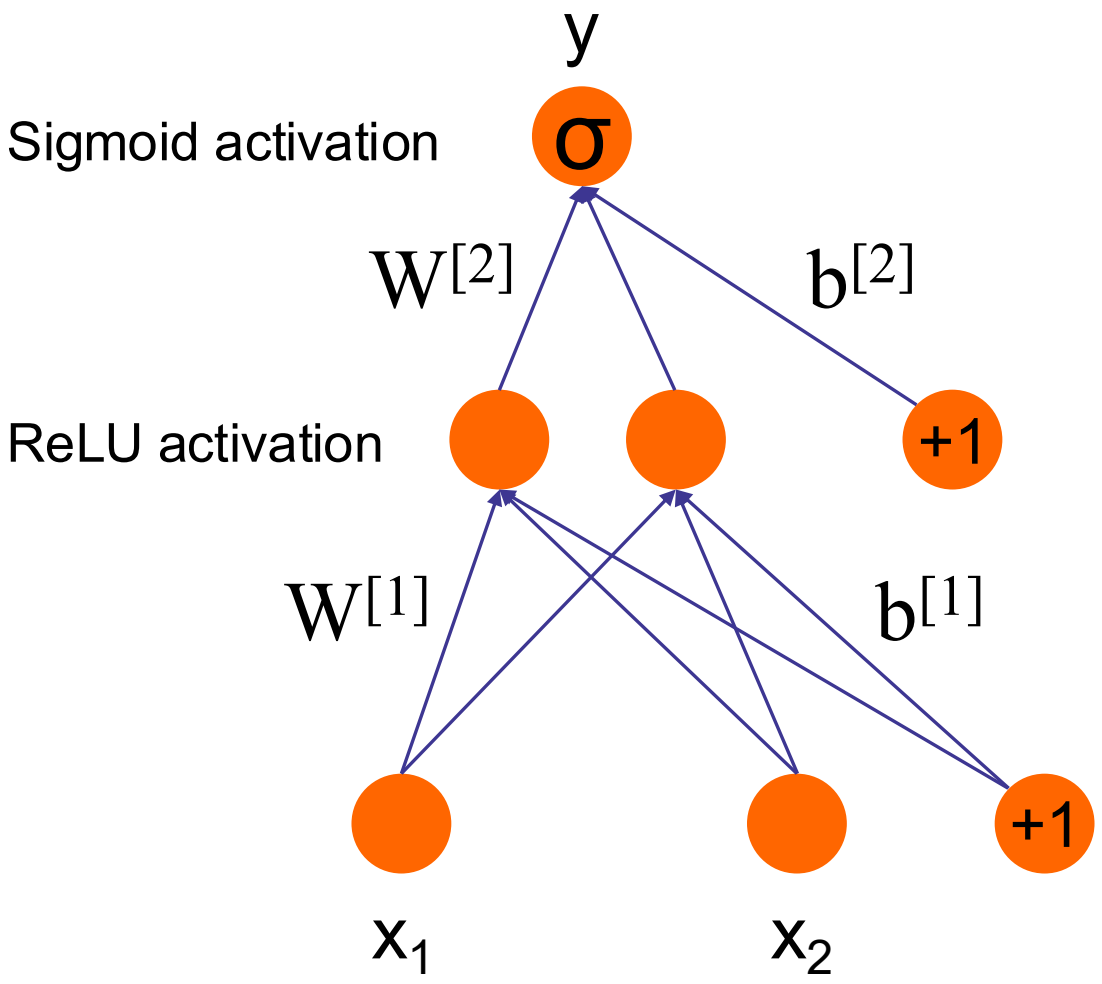
Example: L(a, b, c) = c(a + 2b)

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e}\frac{\partial e}{\partial a}$$
$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e}\frac{\partial e}{\partial d}\frac{\partial d}{\partial b}$$

$$L = ce : \quad \frac{\partial L}{\partial e} = c, \frac{\partial L}{\partial c} = e$$
$$e = a + d : \quad \frac{\partial e}{\partial a} = 1, \frac{\partial e}{\partial d} = 1$$
$$d = 2b : \quad \frac{\partial d}{\partial b} = 2$$



Backward Differentiation on a Two Layer Network



$$
\begin{aligned}
z^{[1]} &= W^{[1]}\mathbf{x} + b^{[1]} \\
a^{[1]} &= \mathrm{ReLU}(z^{[1]}) \\
z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} \\
a^{[2]} &= \sigma(z^{[2]}) \\
\hat{y} &= a^{[2]}
\end{aligned}
$$

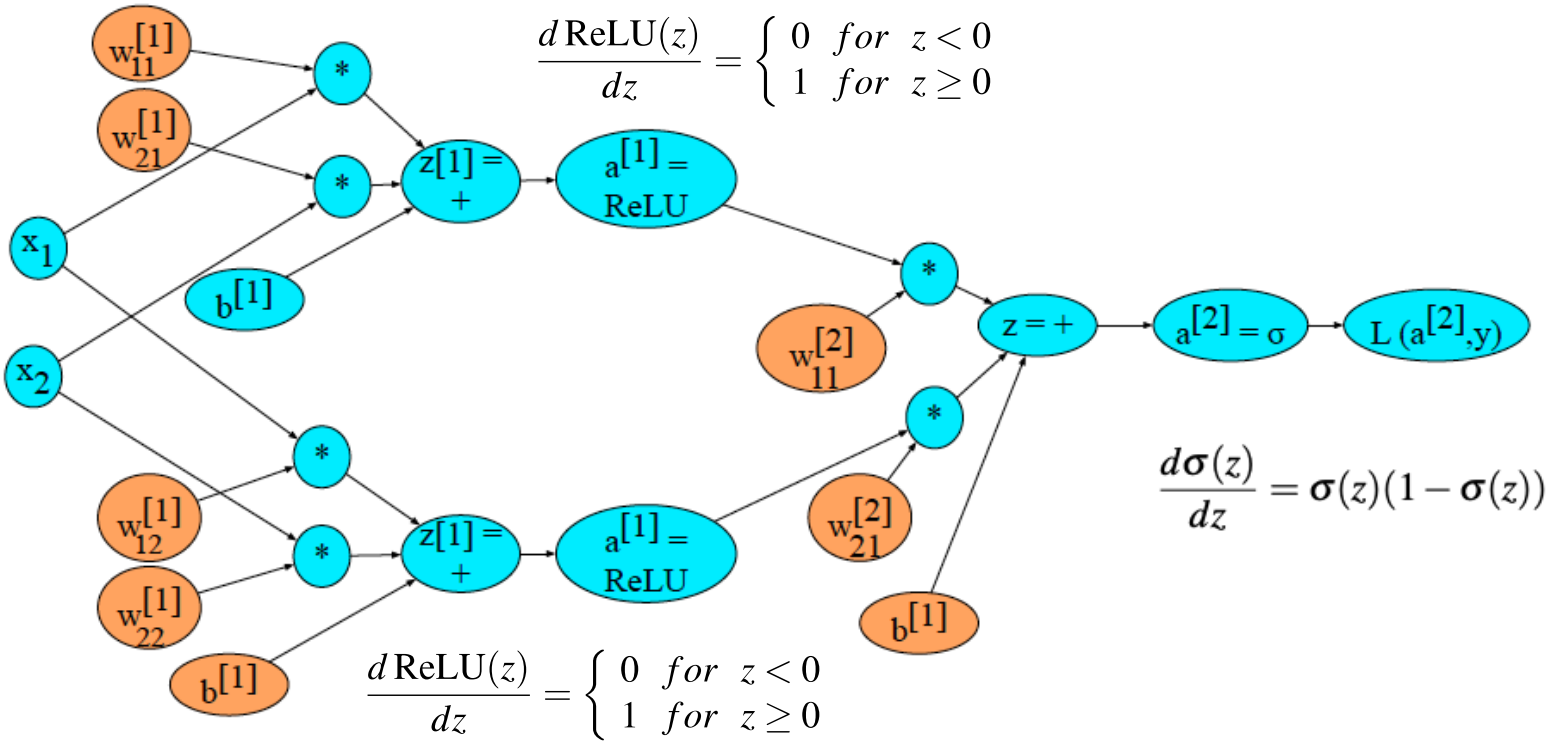
Backward Differentiation on a Two Layer Network

$$
\begin{aligned}
z^{[1]} &= W^{[1]}\mathbf{x} + b^{[1]} \\
a^{[1]} &= \text{ReLU}(z^{[1]}) \\
z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} \\
a^{[2]} &= \sigma(z^{[2]}) \\
\hat{y} &= a^{[2]}
\end{aligned}
$$

$$
\frac{d\sigma(z)}{dz} = \sigma(z)(1-\sigma(z))
$$

$$
\frac{d\,\text{ReLU}(z)}{dz} = \begin{cases} 0 & for \quad z < 0 \\ 1 & for \quad z \geq 0 \end{cases}
$$

Backward Differentiation on a Two Layer Network



Starting Off the Backward Pass: $\frac{\partial L}{\partial z}$

(I'll write a for $a^{[2]}$ and z for $z^{[2]}$)

$$
\begin{aligned}
z^{[1]} &= W^{[1]}\mathbf{x} + b^{[1]} \\
a^{[1]} &= \mathrm{ReLU}(z^{[1]}) \\
z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} \\
a^{[2]} &= \sigma(z^{[2]}) \\
\hat{y} &= a^{[2]}
\end{aligned}
$$

$$L(\hat{y}, y) = -(\mathrm{y}\log(\hat{y}) + (1-y)\log(1-\hat{y}))$$

$$L(a, y) = -(\mathrm{y}\log a + (1-y)\log(1-a))$$

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial z}$$

$$\frac{\partial L}{\partial a} = -\left(\left(y\frac{\partial \log(a)}{\partial a}\right) + (1-\mathrm{y})\frac{\partial \log(1-a)}{\partial a}\right)$$

$$= -\left(\left(y\frac{1}{a}\right) + (1-\mathrm{y})\frac{1}{1-a}(-1)\right)$$

$$\frac{\partial a}{\partial z} = \left(\frac{y}{a} + \frac{y-1}{1-a}\right)$$

$$\frac{\partial L}{\partial z} = -\left(\frac{y}{a} + \frac{y-1}{1-a}\right)a(1-a) = a - y$$

Summary

- For training, we need the derivative of the loss with respect to weights in early layers of the network
  - But loss is computed only at the very end of the network!
- Solution: **backward differentiation**
- Given a computation graph and the derivatives of all the functions in it we can automatically compute the derivative of the loss with respect to these early weights